Тема 24: XML — структуриране, валидация, обработка и представяне.

ТЕОРИЯ

1. Добре-структуриран XML (Well-Formed XML)

XML (eXtensible Markup Language) = език за маркиране за структурирани документи, четими от хора и машини.

Основни компоненти

Компонент	Описание	Пример
Таг	Символи < и > + име	`<book>`
Отварящ таг	`<tag>`	`<person>`
Затварящ таг	`</tag>`	`</person>`
Елемент	Двойка тагове + съдържание	`<name>Иван</name>`
Атрибут	Двойка име=стойност в отварящ таг	`gender="male"`
Текстово съдържание	Текст вътре в елемент	`Иван`
Коментар	`<!-- текст -->`	

```xml
<?xml version="1.0" encoding="UTF-8"?>
<library>
  <book id="1" genre="fiction">
    <title>Под игото</title>
    <author>Иван Вазов</author>
    <year>1894</year>
  </book>
</library>
```

Синтактични правила — Елементи

- Всеки отварящ таг → **задължителен** затварящ (или `<tag/>` за празни)
- Елементите **не се припокриват**: `<a><b></a></b>` е невалидно
- **Case-sensitive**: `<Person>` ≠ `<person>`
- Имената започват с буква или `_` ; допускат букви, цифри, `_` , `-` , `.`
- Имената **НЕ** могат да започват с `xml` , `XML` , `Xml` и варианти
- Само **един корен** (`root element`)

Синтактични правила — Атрибути

- Всеки атрибут **трябва да има стойност** (дори `""`)
- Стойността се огражда с `"..."` или `'...'` (не смесвай)
- **Без дублиращи** атрибути в един елемент
- Разделител между атрибути = интервал

Специални символи (escaping)

Символ	Entity	Употреба
<	`<`	Резервиран за тагове
>	`>`	
&	`&`	Начало на entity reference
"	`"`	В стойности на атрибути
'	`'`	В стойности на атрибути

XML Пространства от имена (Namespaces)

Използват се за **разграничаване** на елементи с еднакви имена от различни схеми.

```
<root xmlns:books="http://example.com/books"
      xmlns:db="http://example.com/database">
  <books:title>XML Programming</books:title>
  <db:title>Database Design</db:title>
</root>
```

---

2. XML валидация чрез DTD (Document Type Definition)

Цели на валидирането

- **Верификация** на структурата — документът следва предварително дефинирани правила
- **Съвместимост** — приложенията очакват конкретна структура

Видове DTD

- **Вътрешен (inline)**: дефиниран в самия XML документ
- **Външен**: в отделен `.dtd` файл, референ в XML-а

```
<!-- Вътрешен DTD -->
<?xml version="1.0"?>
<!DOCTYPE newspaper [
  <!ELEMENT newspaper (article+)>
  ...
]>

<!-- Външен DTD -->
<!DOCTYPE newspaper SYSTEM "newspaper.dtd">
```

Деклариране на елементи

```
<!ELEMENT element_name (content_model)>
```

Тип съдържание	Синтаксис	Описание
Елементно	`(child1, child2)`	Съдържа точно тези деца в реда

Смесено	`(#PCDATA|elem)*`	Текст или елементи; `#PCDATA` трябва да е пръв
Празно	`EMPTY`	Елементът няма съдържание
Произволно	`ANY`	Всякакво съдържание

Индикатори за брой срещания:

Индикатор	Значение
(нищо)	Точно 1 път
`?`	0 или 1 пъти
`*`	0 или повече пъти
`+`	1 или повече пъти

Групиране: `(a, b)` = последователност; `(a | b)` = избор

Деклариране на атрибути

```
<!ATTLIST element_name
  attr_name attr_type default_value>
```

Типове атрибути:

Тип	Описание
`CDATA`	Текстова стойност (не се парсва)
`ID`	Уникален идентификатор на елемента
`IDREF`	Референция към стойност на ID атрибут
`IDREFS`	Списък от IDREF стойности (разделени с интервал)
`ENTITY`	Референция към external entity
`(a|b|c)`	Изброен тип — само тези стойности

Стойности по подразбиране:

Стойност	Значение
`#REQUIRED`	Атрибутът е задължителен
`#IMPLIED`	Атрибутът е незадължителен (няма стойност по подразбиране)
`#FIXED "val"`	Стойността е фиксирана
`"value"`	Стойност по подразбиране

ENTITY и NOTATION

```
<!ENTITY newspaper "Financial Times">          <!-- текстова entity -->
<!ENTITY logo SYSTEM "logo.jpg" NDATA jpeg>   <!-- external unparsed entity -->
<!NOTATION jpeg PUBLIC "image/jpeg">           <!-- нотация за непарсвани данни -->
```

Пример за DTD

```
<?xml version="1.0"?>
<!DOCTYPE newspaper [
  <!ELEMENT newspaper (article+)>
  <!ELEMENT article (heading, body, notes?)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
  <!ELEMENT notes (#PCDATA)>
  <!ATTLIST article
    id     ID    #REQUIRED
    author CDATA #REQUIRED
    date   CDATA #IMPLIED>
  <!ENTITY pub "Sofia Press">
  <!NOTATION jpeg PUBLIC "image/jpeg">
  <!ENTITY logo SYSTEM "logo.jpg" NDATA jpeg>
]>
```

---

3. XML Schema (XSD)

XML Schema = XML документ, описващ структурата на друг XML документ. По-мощен наследник на DTD.

Основни елементи на XSD

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book" type="bookType"/>

  <xs:complexType name="bookType">
    <xs:sequence>
      <xs:element name="title"  type="xs:string"/>
      <xs:element name="author" type="xs:string"/>
      <xs:element name="year"   type="xs:integer"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:ID" use="required"/>
  </xs:complexType>
</xs:schema>
```

Организация на съдържание

Елемент	Описание
`<xs:sequence>`	Точно в указания ред
`<xs:all>`	Присъстват, в произволен ред

`<xs:choice>`	Точно един от изброените

Честота (аналог на DTD индикатори):

```
<xs:element name="item" minOccurs="0" maxOccurs="unbounded"/>
<!-- minOccurs=0, maxOccurs=1 → ? -->
<!-- minOccurs=1, maxOccurs=1 → (нищо) -->
<!-- minOccurs=0, maxOccurs=unbounded → * -->
<!-- minOccurs=1, maxOccurs=unbounded → + -->
```

Прости типове данни

```
<xs:simpleType name="gradeType">
  <xs:restriction base="xs:decimal">
    <xs:minInclusive value="2.00"/>
    <xs:maxInclusive value="6.00"/>
    <xs:fractionDigits value="2"/>
  </xs:restriction>
</xs:simpleType>
```

Фасети (Facets) — ограничаващи

Фасет	Описание
`minInclusive` / `maxInclusive`	Диапазон (включително)
`minExclusive` / `maxExclusive`	Диапазон (изключително)
`length`	Точна дължина
`minLength` / `maxLength`	Диапазон на дължина
`pattern`	Регулярен израз
`enumeration`	Изброени допустими стойности
`totalDigits`	Общ брой цифри (числа)
`fractionDigits`	Брой цифри след десетичната запетая

Сравнение DTD vs XML Schema

Критерий	DTD	XML Schema
Синтаксис	Специфичен (не XML)	XML синтаксис
Типове данни	Ограничени	Богати (xs:integer, xs:date...)
Пространства от имена	Лошо поддържани	Пълна поддръжка
Наследяване	Не	Да
Ограничения (фасети)	Минимални	Богати

Четимост	По-компактен	По-verbose, но по-ясен
Мощност	Базова	Много по-мощен

---

4. XSLT и XPath

XPath — навигация в XML дървото

XPath = език за адресиране на части от XML документ. Представя документа като **дърво от възли**.

Видове възли: корен, елемент, атрибут, текст, коментар, processing instruction, CDATA

```
/                       → корен на документа
/library/book           → всички book директно под library (абсолютен)
book/author             → относителен от текущия контекст
//book                  → всички book навсякъде в документа
book[@genre='fiction'] → book с атрибут genre="fiction"
book[1]                 → първият book
book[last()]            → последният book
@id                     → атрибутът id на текущия елемент
..                      → родителят
.                       → текущият елемент
*                       → всички деца елементи
```

XSLT — трансформация на XML

XSLT = трансформира XML в HTML, текст или друг XML чрез шаблони.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <!-- Шаблон за корена -->
  <xsl:template match="/">
    <html>
      <body>
        <table>
          <xsl:for-each select="library/book">
            <tr>
              <td><xsl:value-of select="title"/></td>
              <td><xsl:value-of select="author"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

Ключови XSLT елементи:

Елемент	Описание

`<xsl:stylesheet>`	Корен на XSLT документа
`<xsl:template match="XPath">`	Шаблон за конкретни възли
`<xsl:apply-templates>`	Прилага шаблони рекурсивно
`<xsl:value-of select="XPath">`	Извежда текстова стойност
`<xsl:for-each select="XPath">`	Итерация по множество възли
`<xsl:if test="condition">`	Условна логика
`<xsl:choose>`/`<xsl:when>`/`<xsl:otherwise>`	switch-like логика
`<xsl:sort select="XPath">`	Сортиране
`<xsl:variable>`	Дефиниция на променлива
`<xsl:element>` / `<xsl:attribute>`	Динамично създаване на елементи

Задаване в XML: `<?xml-stylesheet type="text/xsl" href="style.xsl"?>`

---

5. DOM и SAX

DOM (Document Object Model)

DOM = представя XML като **дървовидна структура в паметта**. Позволява навигация и модификация.

Основни интерфейси:

Интерфейс	Описание
`Document`	Целият документ (корен на дървото)
`Node`	Базов интерфейс; всеки възел в дървото
`Element`	XML елемент; наследява Node
`Attr`	Атрибут с `name` и `value`
`Text`	Текстово съдържание; винаги листо
`Comment`	Коментар
`NodeList`	Колекция от Node обекти
`CharacterData`	Методи за текстообработка
`CDATASection`	CDATA секция
`DocumentType`	DTD информация

Пример (JavaScript):

```
const doc = parser.parseFromString(xmlStr, "text/xml");
const books = doc.getElementsByTagName("book");
```

```
const title = books[0].getElementsByTagName("title")[0].textContent;
```

SAX (Simple API for XML)

SAX = **събитийно базиран** парсър. Не зарежда целия документ в памет — генерира events.

Основни интерфейси/handlers:

Интерфейс	Описание
`ContentHandler`	Основни events: startDocument, endDocument, startElement, endElement, characters
`ErrorHandler`	Три нива: `warning`, `error`, `fatalError`
`DTDHandler`	Events свързани с DTD
`EntityResolver`	Резолвиране на external entities
`XMLReader`	Главен парсер интерфейс

SAX Events (ред при парсиране):

```
startDocument
  startElement("library")
    startElement("book", {id="1"})
      startElement("title")
        characters("Под игото")
      endElement("title")
    endElement("book")
  endElement("library")
endDocument
```

Сравнение DOM vs SAX

Критерий	DOM	SAX
Подход	Цяло дърво в паметта	Събитийно, поточно
Памет	Много (целият документ)	Малко
Скорост	По-бавен (начално зареждане)	По-бърз
Навигация	Произволен достъп	Само напред
Модификация	Да	Не
Подходящ за	Малки/средни документи, промени	Големи документи, четене
Сложност	По-прост за използване	По-сложен

КАК СЕ РЕШАВАТ ЗАДАЧИ

> *Типичната задача: По описание на обекти и връзки → XML документ + DTD или XML Schema.*

Стъпка по стъпка: XML + DTD

Задача: Хотелски каталог — хотели принадлежат на хотелски вериги; предлагат услуги.

Стъпка 1: Идентифицирай обектите и връзките

```
HotelCatalog
  ├── HotelChain (0 или повече)
  │     ├── ChainName (задължително)
  │     └── ChainOwner (незадължително)
  ├── Hotel (1 или повече)
  │     ├── ChainID → реферира HotelChain (IDREF)
  │     ├── HotelName, HotelAddress, HotelImg
  │     └── Service (1 или повече, вградени)
  └── Service (0 или повече, споделени)
        ├── ServiceID (ID)
        ├── ServiceName
        └── ServiceDescr (незадължително)
```

Стъпка 2: DTD

```
<?xml version="1.0"?>
<!DOCTYPE HotelCatalog [
  <!ELEMENT HotelCatalog (HotelChain*, Hotel+, Service*)>
  <!ATTLIST HotelCatalog Version CDATA #REQUIRED>

  <!ELEMENT HotelChain (ChainName, ChainOwner?)>
  <!ATTLIST HotelChain id ID #REQUIRED>
  <!ELEMENT ChainName (#PCDATA)>
  <!ELEMENT ChainOwner (#PCDATA)>

  <!ELEMENT Hotel (HotelName, HotelAddress, HotelImg, Service+)>
  <!ATTLIST Hotel
    ChainID   IDREF  #IMPLIED
    ServiceID IDREFS #IMPLIED>
  <!ELEMENT HotelName (#PCDATA)>
  <!ELEMENT HotelAddress (#PCDATA)>
  <!ELEMENT HotelImg (#PCDATA)>

  <!ELEMENT Service (ServiceName, ServiceDescr?)>
  <!ATTLIST Service
    id       ID          #IMPLIED
    Included (YES | NO)  #IMPLIED>
  <!ELEMENT ServiceName (#PCDATA)>
  <!ELEMENT ServiceDescr (#PCDATA)>

  <!NOTATION jpg PUBLIC "image/jpeg">
  <!ENTITY hotelImage SYSTEM "hotel.jpg" NDATA jpg>
]>
```

Стъпка 3: XML документ

```xml
<?xml version="1.0" encoding="UTF-8"?>
<HotelCatalog Version="1.0">

  <HotelChain id="chain1">
    <ChainName>Luxury Hotels</ChainName>
    <ChainOwner>John Doe</ChainOwner>
  </HotelChain>

  <Hotel ChainID="chain1" ServiceID="s1 s2">
    <HotelName>Grand Palace</HotelName>
    <HotelAddress>Main Street 123, Sofia</HotelAddress>
    <HotelImg>&hotelImage;</HotelImg>
    <Service>
      <ServiceName>Room Service</ServiceName>
      <ServiceDescr>24/7 Room Service</ServiceDescr>
    </Service>
    <Service>
      <ServiceName>Free Breakfast</ServiceName>
    </Service>
  </Hotel>

  <Service id="s1" Included="YES">
    <ServiceName>Room Service</ServiceName>
    <ServiceDescr>24/7 Room Service</ServiceDescr>
  </Service>
  <Service id="s2" Included="YES">
    <ServiceName>Free Breakfast</ServiceName>
  </Service>

</HotelCatalog>
```

Стъпка по стъпка: XML Schema (XSD) вариант

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="HotelCatalog">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="HotelChain" minOccurs="0" maxOccurs="unbounded"
                    type="HotelChainType"/>
        <xs:element name="Hotel" minOccurs="1" maxOccurs="unbounded"
                    type="HotelType"/>
        <xs:element name="Service" minOccurs="0" maxOccurs="unbounded"
                    type="ServiceType"/>
      </xs:sequence>
      <xs:attribute name="Version" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>

  <xs:complexType name="HotelChainType">
```

```
    <xs:sequence>
      <xs:element name="ChainName"  type="xs:string"/>
      <xs:element name="ChainOwner" type="xs:string" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:ID" use="required"/>
  </xs:complexType>

  <xs:complexType name="ServiceType">
    <xs:sequence>
      <xs:element name="ServiceName"  type="xs:string"/>
      <xs:element name="ServiceDescr" type="xs:string" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="id"       type="xs:ID"/>
    <xs:attribute name="Included" type="IncludedEnum"/>
  </xs:complexType>

  <xs:simpleType name="IncludedEnum">
    <xs:restriction base="xs:string">
      <xs:enumeration value="YES"/>
      <xs:enumeration value="NO"/>
    </xs:restriction>
  </xs:simpleType>

</xs:schema>
```

Бърза справка

Концепция	Ключ
Well-formed	Затворени тагове, без припокриване, 1 корен
`#PCDATA`	Текстово съдържание в DTD
`+` / `*` / `?`	DTD честота: 1+, 0+, 0 или 1
`ID`	Уникален ключ на елемент
`IDREF`	Референция към ID
`IDREFS`	Списък от IDREF (разделени с интервал)
`#REQUIRED`	Атрибутът е задължителен
`#IMPLIED`	Незадължителен без default
`xs:sequence`	Деца в точен ред
`xs:choice`	Само едно от децата
`xs:restriction`	Ограничава базов тип (фасети)
`minOccurs/maxOccurs`	XSD честота (unbounded = ∞)
XPath `//`	Навсякъде в документа

XPath `@attr`	Атрибут
XPath `[cond]`	Филтриране
DOM	Цяло дърво в памет; навигация и промени
SAX	Събитийно; за големи документи